IMPLEMENTATION OF RISC-V PROCESSOR

P.Saiprathyusha and C.Chandrasekhar

Department of ECE, Sri Venkateswara College of Engineering, Tirupati, India
Professor, Department of ECE, Sri Venkateswara College of Engineering, Tirupati, India

Abstract. This work focuses on implementation/designing the RISC-V Processor with optimized pipeline throughput, cache hit rate, and dynamic instruction scheduling to enhance the processing speed and energy efficiency. RISC-V extension used to support the tasks in AI, signal processing and cryptography. Design of processor will be implemented by using Verilog/VHDL and simulation tools later it will be tested on FPGA hardware. This project in designing to improve the performance mainly used for high-performance application.

1 Introduction

During 1970s, Very Large-Scale Integration (VLSI) Design was emerged and brings a drastic changes and advancements in semiconductor and communication technologies. These developments brings out an innovation of microprocessor design and electronic computing. This rapid progress may get advancements in areas like aviation, healthcare, mobile devices, and the automotive sector of computing.

By 2030, it is estimated more than 22.7 billion devices will be interlinked through IOT networks (W. Wang et al., 2021), it brings out the future advancements or innovations in sectors like smart cities, self-driving vehicles and space exploration technologies.

VLSI, Very large scale integration is defines as a method of designing the intergrated circuits by adding thousands of transistors onto a single chip. Before VLSI, ICs have very few functionalities . After VLSI Technology, Engineers can consolidate the crucial components such as the Central Processing Unit (CPU), Read-Only Memory (ROM), Random Access Memory (RAM), and other logical units onto a single chip. Modern innovations such as lowbit-rate, high-definition video and cellular networks, provides enhanced applications, processing capabilities and portability to users. This evolution is expected to persist, significantly shaping the VLSI and system design in the upcoming years.

Today's Processors are maily based on two key architectural paradigms. They are : Complex Instruction Set Computing (CISC) and Reduced Instruction Set Computing (RISC). CISC architecture is defined as an extensive set of instructions which are capable of executing multiple step operations within a single command. This approach is flexible and it is also reduces the number of instructions needed for a program so that it minimizes the complexity of a program. Making it suitable for scenarios where backward compatibility with older software or specialized instructions is crucial, such as in personal computers and certain

© The Authors, published by EDP Sciences. This is an open access article distributed under the terms of the Creative Commons Attribution License 4.0 (https://creativecommons.org/licenses/by/4.0/).

highperformance computing tasks. The Design of CISC often leads to higher power consumption and complex circuitry because of this production costs and energy demands increases.

The other key architectural paradigm is Reduced Instruction Set Computing (RISC). This RISC Processor mainly focuses on simplicity and efficiency. It's uses a smaller set of instructions unlike CISC because of this smaller set of instruction its executed rapidly. This architecture enables the faster data processing, speeds, better energy efficiency, and easier opeimization for parallel processing. Due to this advantages , RISC architecture design becomes famous in most of the devices where power efficiency and quick processing is needed, like smartphones, tablets, and other mobile or electronic devices. Both CISC and RISC have their own features and properties both have advantages and disadvantages, However, making research into enhancing their strengths a valuable endeavor for both academia and industry as demand for higher performance computing solutions grows.

RISC-V was introduced in 2010 by the University of California, Berkeley, as an open and freely available Instruction Set Architecture (ISA) rooted in the principles of RISC. It is opensource design which removes the licensing costs, key obstacles in the semiconductor industry. This allows companies to tailor and deploy RISC-V to meet their specific needs without being limited by proprietary constraints. Consequently, RISC-V has gained significant attention across various sectors, including embedded systems, mobile technology, data centre, and artificial intelligence.

As the demand for more powerful and efficient computing grows, RISC-V is likely to become even more important. Its ability to offer low-cost, customizable, and high-performance solutions makes it a key player in the semiconductor and computing industries. RISC-V is paving the way for new technologies in everything from everyday electronics to self-driving systems and beyond, helping to shape the future of technology. In future RISC-V processor plays very crucial role in most of the electronic devices which gradually increases the performance of the devices.

2 Methodology

2.1 Background Theories

2.1.1 Instruction Set Architecture (ISA)

The Instruction Set Architecture (ISA) defines the processor commands that can carry out, which is a key part of how a computer is built and works. This ISA have operations (called opcodes) and the data (called operands) which are used to perform tasks. For example, in a MOV instruction, the opcode is "MOV," and the operands tell the processor which data needs to move and where to move, like moving the value 54 in hexadecimal to a register (e.g., AL).

There are two main types of Instruction Set Architectures (ISAs), They are: Reduced Instruction Set Computing (RISC) and Complex Instruction Set Computing (CISC). RISC ISAs have fewer and simpler instructions, which makes them faster and more efficient, making them ideal and suitable for modern processors. CISC ISAs, have more complex instructions that can perform multiple tasks at once, which is helpful for certain specialized applications but harder to optimize and less efficient. One of the popular RISC ISA is ARM, it focuses on simplicity, energy efficiency, and fast execution, making it easier and suitable to optimize across different processors.

2.1.2 5-Stage Pipeline

Traditional processors execute instructions one by one, leading to delays if instructions vary in execution time. Pipelining solves this issue by dividing instruction execution into distinct stages, enabling the concurrent processing of multiple instructions to enhance overall efficiency.

The **5-stage pipeline** is a standard method in modern processors, with each stage handling a specific task sequentially, improving throughput. The five stages are:

1. **Fetch**: The processor retrieves the instruction from memory.
2. **Decode**: It analyses the instruction to identify the operation and operands.
3. **Execute**: The operation, like arithmetic or logical tasks, is performed.
4. **Memory**: The processor accesses memory as needed.
5. **Writeback**: Results are saved to the register.

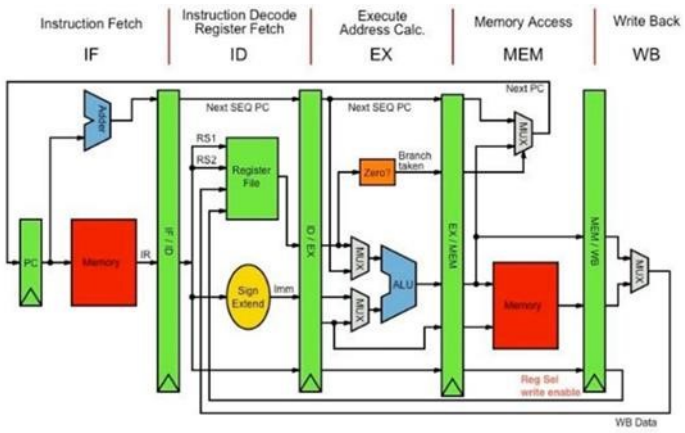


Fig 1: 5-Stage Pipeline

3 Implementation

3.1 Implementation of Design

The processor comprises several components, including the ALU, ALU decoder (aludec), main decoder (maindec), instruction memory (imem), data memory (dmem), pipeline registers, multiplexers, and forwarding units. Key elements of the datapath are summarized below.

3.1.1 Arithmetic Logic Unit (ALU)

The ALU handles arithmetic and logical operations on binary data, directed by signals from the ALU decoder. It retrieves data from the pipeline register (ID_IEx) and delivers the output to ALUResults.This RISC-V ALU implementation supports nine instructions, including addition, subtraction, bitwise operations (AND, OR, XOR), and shifts.

```
/*
    Name: ALU Unit
    Description: Receives control signals from the ALU Decoder and performs the
operations
*/


module alu(input logic [31:0] SrcA,
           input logic [31:0] SrcB,
           input logic [3:0] ALUControl ,
           output logic  [31:0] ALUResult,
           output logic Zero, Sign);

logic [31:0] Sum;
logic Overflow;

assign Sum = SrcA + (ALUControl[0] ? ~SrcB : SrcB) + ALUControl[0];  // sub using
1's complement
assign Overflow = ~(ALUControl[0] ^ SrcB[31] ^ SrcA[31]) & (SrcA[31] ^ Sum[31])
& (~ALUControl[1]);

assign Zero = ~(|ALUResult);
assign Sign = ALUResult[31];


always_comb
        casex (ALUControl)
                4'b000x: ALUResult = Sum;              // sum or diff
                4'b0010: ALUResult = SrcA & SrcB;   // and
                4'b0011: ALUResult = SrcA | SrcB;   // or
                4'b0100: ALUResult = SrcA << SrcB;  // sll, slli
```

Fig 2: Implementation of ALU in Verilog code

3.1.2 ALU Decoder (aludec)

The ALU decoder interprets instructions to identify the required ALU operation. It combines inputs (ALUOp, funct3, funct7b5, opb5) to generate control signals. For instance, RTypeSub distinguishes between R-type ADD and SUB instructions by applying bitwise operations to funct7b5 and opb5.

```
/*
    Name: ALU Decoder
    Description: Receives control signal from the Main Decoder Unit and
    determines the type of operation that has to be performed by the ALU

*/


module aludec(input logic opb5,
    input logic [2:0] funct3,
    input logic funct7b5,
    input logic [1:0] ALUOp,
    output logic [3:0] ALUControl);

logic RtypeSub;
assign RtypeSub = funct7b5 & opb5; // TRUE for R-type subtract
always_comb
    case(ALUOp)
        2'b00: ALUControl = 4'b0000; // addition
        2'b01: ALUControl = 4'b0001; // subtraction
        default: case(funct3) // R-type or I-type ALU
            3'b000: if (RtypeSub)
                ALUControl = 4'b0001; // sub
            else
                ALUControl = 4'b0000; // add, addi
            3'b001: ALUControl = 4'b0100; // sll, slli
            3'b010: ALUControl = 4'b0101; // slt, slti
            3'b011: ALUControl = 4'b1000; // sltu, sltiu
            3'b100: ALUControl = 4'b0110; // xor, xori
            3'b101: if (~funct7b5)
                ALUControl = 4'b0111;  // srl
            else
                ALUControl = 4'b1111;  // sra
            3'b110: ALUControl = 4'b0011; // or, ori
            3'b111: ALUControl = 4'b0010; // and, andi
            default: ALUControl = 4'bxxxx; // ???
            endcase
    endcase

endmodule
```

Fig 3: Implementation of aludec in Verilog code

3.1.3 Main Decoder (maindec)

The main decoder generates control signals (RegWrite, ALUSrc, MemWrite, etc.) based on the instruction's opcode. These control signals guide the data flow within the datapath through multiplexers, enabling different data processing and memory operations as needed.

The job of Main Decoder is to figure out what the instructions means and what actions needs to be performed. It takes the operation code and breaks it down, deciding the which parts of the processor should be activated to carry out the task.

Main Decoder is used to translate the instruction into signals which controls the processor. It ensures whether the operations are carried out is right or wrong.

```
logic [13:0] controls;
assign {RegWrite, ImmSrc, ALUSrcA, ALUSrcB, MemWrite, ResultSrc, Branch, ALUOp,
Jump} = controls;

always_comb
    case(op)
    // RegWrite_ImmSrc_ALUSrcA_ALUSrcB_MemWrite_ResultSrc_Branch_ALUOp_Jump
        7'b0000011: controls = 14'b1_000_0_01_0_01_0_00_0; // lw
        7'b0100011: controls = 14'b0_001_0_01_1_00_0_00_0; // sw
        7'b0110011: controls = 14'b1_xxx_0_00_0_00_0_10_0; // R-type
        7'b1100011: controls = 14'b0_010_0_00_0_00_1_01_0; // B-type
        7'b0010011: controls = 14'b1_000_0_01_0_00_0_10_0; // I-type ALU
        7'b1101111: controls = 14'b1_011_0_00_0_10_0_00_1; // jal
        7'b0000000: controls = 14'b0_000_0_00_0_00_0_00_0; // for default values
on reset

        default:     controls = 14'bx_xxx_x_xx_x_xx_x_xx_x; // instruction not
implemented
    endcase
endmodule
```

Fig 4: Implementation of maindec in Verilog code

3.1.4 Pipeline Register

Pipeline registers are temporary storage elements in a processor's pipeline that facilitate data transfer between instruction execution stages, ensuring synchronization and controlled data flow. A 5-stage pipeline contains four registers: Fetch-Decode Register, Decode-Execute Register, Execute-Memory Register, and Memory-Writeback Register, each named to represent the stages they divide.

3.1.5 Instruction Memory (imem)

Instruction memory holds program instructions for the processor. The PCF input, controlled by the program counter, fetches instructions in sequence, incrementing by 4 bytes per instruction.

4 Result

4.1 Signal Behaviour of Instruction Memory (imem)

The **Instruction Storage Unit (imem)** controls the instruction flow based on the **Instruction Address Fetch** input, which increments by 4 bytes for each instruction. Figure 4.3 shows a simulation waveform, and the relationship between **PCF** and the fetched instruction. Each clock cycle, the **PCF** increases by 4, fetching a different instruction

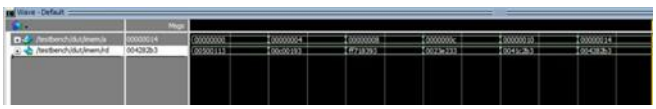


Fig 5: Timing Diagram of imem

4.2 Signal Behaviour Analysis of Data Memory (dmem)

The **Data Cache Module (dmem)** stores data with write enabled by **MemWrite**. Figure 4.4 captures the simulation waveform. The **sw instruction** is analyzed, particularly instruction **0x0471AA23**. After three cycles post-fetch, the data reaches the memory stage, where it writes data if **MemWrite** is set. The functionality of **dmem** is verified.

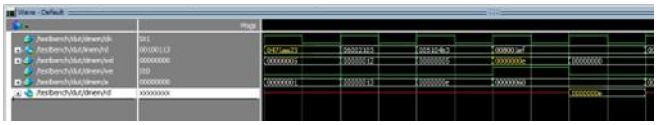


Fig 6: Simulation waveform of dmem

4.3 ALU Timing Waveform Examination

The **alu** performs arithmetic and logical operations using inputs from the ID_IEx stage (SrcA, SrcB) and control signals from the **ALU decoder**. Which include operations like **OR, XOR, ADD, BEQ**, and **SLT**.



Fig 7: Simulation Waveform of alu

4.4 Signal Behaviour Analysis of Core Decoder (maindec)

The **Core Decoder** generates control signals from the instruction opcode. Figure 4.8 shows the simulation results for opcodes **0x0110011** (R-type) and **0x1100011** (B-type). Control signals for various instruction types are summarized.

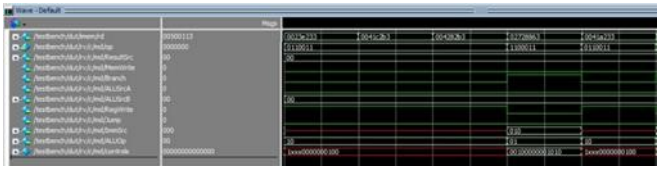


Fig 8: Core Decoder Simulation Trace

References

1. R. Urquhart, "What Does RISC-V Stand For? A Brief History of the Open ISA," *Semiconductor Engineering: Systems & Design*, Opinion, Mar. 29, 2021.
2. D. Bhandarkar and D. W. Clark, "Performance from Architecture: Comparing a RISC and a CISC with Similar Hardware Organization," in *Proc. 4th Int'l Conf. ASPLOS*, Santa Clara, CA, USA, Apr. 8–11, 1991.
3. M. N. Topiwala and N. Saraswathi, "Implementation of a 32-bit MIPS Based RISC Processor Using Cadence," *2014 IEEE Int'l Conf. Adv. Commun., Control and*

Computing Technologies, 2014, pp. 979–983, doi: 10.1109/ICACCCT.2014.7019240.

4. A. Kulshreshtha, A. Moudgil, A. Chaurasia, and B. Bhushan, "Analysis of 16-Bit and
5. 32-Bit RISC Processors," in *Proc. 2021 7th Int'l Conf. Adv. Comput. Commun. Syst. (ICACCS)*, vol. 1, 2021, pp. 1318–1324.
6. S. Islam, D. Chattopadhyay, M. K. Das, V. Neelima, and R. Sarkar, "Design of HighSpeed Pipelined Execution Unit of 32-bit RISC Processor," in *Proc. 2006 IEEE India Conf.*, 2006, pp. 1–5.
7. S. S. Khairullah, "Realization of a 16-bit MIPS RISC Pipeline Processor," in *Proc. 2022 Int'l Congr. Human-Computer Interact., Optimization, and Robotic Applications (HORA)*, 2022, pp. 1–6.
8. S. M. Al-sudany, A. S. Al-Araji, and B. M. Saeed, "FPGA-Based Multi-Core MIPS Processor Design," *Iraqi J. Comput., Commun., Control Syst. Eng.*, vol. 21, no. 2, 2021.